Übungsblatt 2: Scheduling-Theorie und kooperativer Dispatcher

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Ziele und Ablauf

Sie beschreiben Jobs, prüfen RMS und EDF unter benannten Annahmen und erklären den kooperativen Aufrufring. Grundlage sind Vorlesungen 1–2. RMS bedeutet Rate Monotonic Scheduling, EDF Earliest Deadline First. Für den Aufrufring wird kein Echtzeitbetriebssystem benötigt.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	32 Orientierungspunkte
Transfer und Besprechung	10	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **75 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 90 Minuten Erklärung und 15 Minuten Quiz zum Termin mit 180 Minuten.

Teil A: Papieraufgaben

Gemeinsames Modell und Notation

Sofern nicht anders angegeben: genau ein Prozessor, unabhängige periodische Aufgaben, vollständig präemptierbare Jobs, keine Sperren oder Selbstsuspendierung, kein Freigabejitter. Interrupt-, Auswahl- und Wechselkosten sind null. Ausführungszeiten sind vorgegebene obere Schranken. Diese **Lehrannahmen** sind keine Boardmessung und gelten nicht automatisch für den kooperativen Code in Teil B.

Größe	Bedeutung
C_i	eigener Prozessorbedarf eines Jobs der Aufgabe τ_i
T_i	Abstand periodischer Freigaben
D_i	relative Abschlussfrist ab Freigabe
$r_{i,k}$, $s_{i,k}$, $f_{i,k}$	Freigabe, erster Start, Abschluss des Jobs k
$d_{i,k} = r_{i,k} + D_i$	absolute Deadline
$R_{i,k} = f_{i,k} - r_{i,k}$	Antwortzeit des einzelnen Jobs
R_i	nachgewiesene maximale Antwortzeit im Modell

Task bedeutet wiederkehrende Aufgabe, Job eine einzelne Aktivierung. Zeitintervalle sind halboffen: $[a, b)$ enthält den Anfang, aber nicht den Endpunkt. Abschluss genau auf der Deadline gilt als rechtzeitig.

Aufgabe 1: Ein Job auf der Zeitachse · 8 Minuten · 6 Punkte

Eine Aufgabe hat $T = 10\,\text{ms}$ und $D = 8\,\text{ms}$. Ein Job wird bei $r = 0$ freigegeben. Er wartet bis $s = 2$, rechnet von 2 bis 3, wird von einer anderen Aufgabe verdrängt und rechnet von 5 bis 7 weiter. Alle Zeitpunkte sind in Millisekunden.

1. Zeichnen Sie die Belegung dieses Jobs in $[0, 10)$. Kennzeichnen Sie Freigabe, Start, beide Rechenstücke, Abschluss, absolute Deadline und nächste Freigabe. (3 P)
2. Berechnen Sie eigene Prozessorzeit, Startlatenz und Antwortzeit. Prüfen Sie die Frist. Warum sind beide Rechenstücke derselbe Job? (2 P)
3. Ist die verdrängte Aufgabe von 3 bis 5 Ready oder Blocked? Begründen Sie anhand ihrer Warteursache. (1 P)

Aufgabe 2: RMS-Schranke richtig interpretieren · 8 Minuten · 6 Punkte

Für alle drei Aufgaben gilt $D_i = T_i$:

Aufgabe	C_i in ms	T_i in ms
τ_1	1	4
τ_2	2	6
τ_3	2	12

1. Berechnen Sie $U = \sum_i C_i/T_i$ als Bruch und Prozentwert. (2 P)

2. Vergeben Sie Prioritäten nach RMS, Rate Monotonic Scheduling. Prüfen Sie die hinreichende Liu-und-Layland-Schranke $U \leq n(2^{1/n} - 1)$ für $n = 3$. (2 P)
3. Was würde ein Überschreiten dieser Schranke bei weiterhin $U \leq 1$ bedeuten? Darf der bestandene Test unverändert auf den kooperativen Round-Robin aus Teil B übertragen werden? Begründen Sie beide Antworten. (2 P)

Aufgabe 3: Antwortzeit trotz überschrittener Schranke · 14 Minuten · 8 Punkte

Dieses Set entspricht dem 85-%-Beispiel der Vorlesung; es ist **ein anderes Set als in Aufgabe 2**. Wieder gilt $D_i = T_i$:

Aufgabe	C_i in ms	T_i in ms
τ_1	1	4
τ_2	2	5
τ_3	2	10

Die RTA, Response-Time Analysis oder Antwortzeitanalyse, verwendet für feste Prioritäten:

$$R_i^{(0)} = C_i, \qquad R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j.$$

$hp(i)$ ist die Menge höher priorisierter Aufgaben, $\lceil x \rceil$ rundet auf die nächste ganze Zahl auf. Iterieren Sie bis zu einem unveränderten Wert oder einer Überschreitung der Deadline.

1. Bestimmen Sie U und die RMS-Reihenfolge. Welche Aussage liefert allein die Liu-und-Layland-Schranke? (2 P)
2. Berechnen Sie R_1, R_2 und alle Iterationswerte für R_3 mit Zwischenrechnungen. (4 P)
3. Prüfen Sie jede Frist und erklären Sie, warum $\lceil 5/5 \rceil = 1$ gilt. Darf aus dem Ergebnis „85 % ist unter RMS immer sicher“ folgen? (2 P)

Aufgabe 4: RMS und EDF am selben Set · 12 Minuten · 6 Punkte

Zwei Aufgaben starten synchron bei null:

Aufgabe	C_i in ms	$T_i = D_i$ in ms
τ_1	2	5
τ_2	4	7

EDF bedeutet Earliest Deadline First: Aus den bereiten Jobs wird der mit der frühesten **absoluten** Deadline gewählt. Bei Gleichstand bleibt der laufende Job. Es gilt das gemeinsame präemptive Modell.

1. Berechnen Sie U. Zeichnen Sie unter RMS die Belegung bis 7 ms und zeigen Sie die Fristverletzung. (2 P)
2. Zeichnen Sie unter EDF die Belegung bis 12 ms. Beschriften Sie die Jobs und begründen Sie die Entscheidungen bei 5 und 10 ms mit den absoluten Deadlines. (3 P)
3. Was bedeutet dieses Ergebnis für den Unterschied zwischen „unter RMS nicht einplanbar“ und „überhaupt nicht machbar“? Begründen Sie die EDF-Aussage für alle periodischen Jobs, nicht nur den gezeichneten Ausschnitt. (1 P)

Aufgabe 5: Kooperative Runden und Jitter · 8 Minuten · 6 Punkte

Hier gilt ausdrücklich ein **anderes Modell**: Drei Schritte A, B und C sind ständig bereit. Der Dispatcher ruft sie zyklisch A, B, C auf. Jeder Schritt kehrt nach seiner Arbeit zurück. Es gibt keine erzwungenen Zeitscheiben und keine periodischen Jobfreigaben. Erster A-Start ist bei null; Aufruf- und Interruptkosten sind null.

A benötigt 1 ms, C ebenfalls 1 ms. Ein Marker ist während der Arbeit von A High. Definieren Sie Pulsbreite $w_k = f_k - s_k$, Startabstand $P_k = s_{k+1} - s_k$ und Peak-to-Peak-Jitter $J_P = \max(P_k) - \min(P_k)$.

1. Bestimmen Sie die ersten fünf A-Starts, Pulsbreite, Startabstände und J_P bei konstant 1 ms für B. Wiederholen Sie für konstant 3 ms für B. Ist die konstante Verlängerung allein schon Jitter? (2 P)
2. B benötigt abwechselnd 1, 3, 1, 3 ms. Bestimmen Sie die ersten fünf A-Starts, die vier Startabstände, Pulsbreite und J_P. (2 P)
3. B kehrt stattdessen nie zurück. Was geschieht mit weiteren A-/C-Aufrufen und mit weiterhin erlaubten Tickinterrupts? Warum ist `wake_at` setzen ohne Rückkehr kein Taskwechsel, und warum liefert eine Wake-Zeit keinen genauen Startzeitpunkt? (2 P)

Transfer: Ein kleiner Wert genügt nicht · 10 Minuten

Diese zusätzliche Klausurübung hat keine weiteren Punkte. Ein präemptives Ein-Prozessor-Modell verwendet unabhängige periodische Jobs, keine Zusatzkosten und synchrone Freigaben bei null. Zwei Aufgaben haben jeweils $C = 2\,\text{ms}$, $T = 10\,\text{ms}$ und $D = 3\,\text{ms}$.

1. Berechnen Sie U. Eine Person behauptet: „Unter 100 % hält EDF jede Frist.“ Prüfen Sie die fehlende Voraussetzung und zeichnen Sie den ersten kritischen Abschnitt.
2. Lässt sich das Problem durch einen anderen Tie-Breaker oder RMS beheben? Begründen Sie mit der benötigten Arbeit bis zur gemeinsamen absoluten Deadline.

Teil B: Praxisfall · 15 Minuten

Praxisfall 2: Aufgewacht heißt noch nicht aufgerufen

Papieralternative, Lehrmodell: Der Ring durchsucht A, B und C in dieser Reihenfolge. Alle sind zunächst Ready. Ein Aufruf läuft bis zur Rückkehr. A arbeitet 1 ms, setzt am Arbeitsende ein Wake-Ziel „jetzt plus 10 ms“ und kehrt sofort zurück. B arbeitet 6 ms, C 1 ms; beide bleiben Ready. Zeit läuft auch während B weiter, wechselt aber keinen Schritt. Verwaltungs- und Interruptkosten sind null. Der erste A-Aufruf beginnt bei null.

1. Verfolgen Sie die ersten sechs Aufrufe. Wann endet As erster Aufruf, wann ist sein Wake-Ziel erreicht und wann beginnt sein zweiter Aufruf?
2. Kennzeichnen Sie A als Blocked oder Ready zwischen den Aufrufen. Warum kann A bei Erreichen seines Ziels B nicht unterbrechen?
3. Würde ein weiterer Tickhandler oder eine nicht zurückkehrende B-Funktion das Problem lösen?

Bei vorbereitetem Board: Die Lehrperson zeigt den Unterschied zwischen `LAB02_SHORT` und `LAB02_SLEEP_A`. Die Gruppe erklärt Verwaltungszustand, Rückkehr und Auswahl. Keine Millisekundendauer wird aus Iterationszahlen abgeleitet.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (32 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesungen 1–2; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*; Liu und Layland (1973) zur RMS-Schranke; die behandelte Antwortzeitanalyse; ST UM2408, RM0399 und PM0214 sowie die Dokumentation der tatsächlich verwendeten STM32-Erweiterung. Vollständiger Anwendungsbaustein und Integrationsanleitung.